

queue 정리

먼저 넣은 값이 먼저 나오는 FIFO 구조입니다.

사용처 (Where)

- 대기열 처리
- BFS
- 작업을 들어온 순서대로 처리할 때
- 순서가 중요한 이벤트 처리

방법 (How to Use)

- push로 뒤에 넣음
- front로 앞 확인
- pop으로 앞 제거
- empty 확인 후 접근

기본 정보

- 필요 헤더: `#include <queue>`
- 기본 선언: `queue<int> q;`

왜 써야 할까? (Why)

- 순서대로 처리한다는 의미가 명확함
- BFS 구현의 기본 도구
- 직접 배열로 head/tail 관리하지 않아도 됨

주요 함수

함수/문법	의미
<code>push(x)</code>	뒤에 추가
<code>pop()</code>	앞 원소 제거
<code>front()</code>	앞 원소 확인
<code>back()</code>	뒤 원소 확인
<code>empty()/size()</code>	상태 확인

기본 코드 예시

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;
    q.push(1);
    q.push(2);

    while (!q.empty()) {
        cout << q.front() << ' ';
        q.pop();
    }
}
```

queue 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
push/pop/front/back	O(1)
검색	직접 지원 안 함
임의 접근	지원 안 함

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: BFS 기본 형태

시작점을 queue에 넣고, 하나씩 꺼내며 주변 노드를 다시 queue에 넣습니다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    vector<vector<int>> graph = {{1,2}, {0,3}, {0}, {1}};
    vector<int> visited(4, 0);
    queue<int> q;

    q.push(0);
    visited[0] = 1;

    while (!q.empty()) {
        int cur = q.front();
        q.pop();
        cout << cur << ' ';

        for (int next : graph[cur]) {
            if (!visited[next]) {
                visited[next] = 1;
                q.push(next);
            }
        }
    }
}
```

처음 배울 때 자주 하는 실수

- empty 확인 없이 front/pop 호출 금지
- 중간 원소 접근이 필요하다면 queue가 맞지 않음
- 우선순위가 필요하다면 priority_queue 사용

비슷한 항목과 차이

- stack은 후입선출
- deque는 앞뒤 접근이 모두 가능
- priority_queue는 우선순위가 높은 값부터 나옴

한 줄 정리: queue은(는) '대기열 처리' 상황에서 먼저 떠올리면 좋은 STL입니다.